Introduction to MLIR

Session 2, Part 2 — Distributing Linalg on the GPU

SC26 Tutorial

Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.

Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **It is a fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain again.

Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **It is a fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain again.
- **The exercise edits one MLIR file, a schedule,** as in Part 1: edit, save, re-run the cell. No C++.

Setup

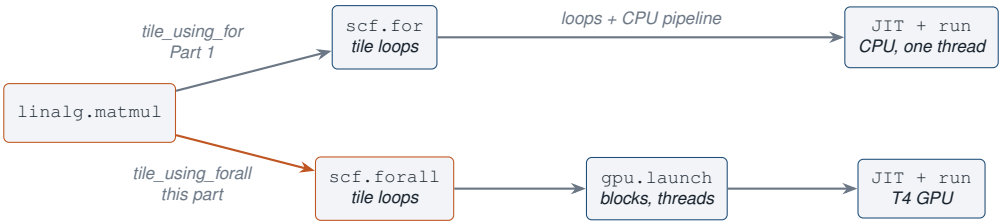
- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **It is a fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain again.
- **The exercise edits one MLIR file, a schedule,** as in Part 1: edit, save, re-run the cell. No C++.
- **One exercise:** distribute matmul along its columns too.

Same Tiling, onto the GPU



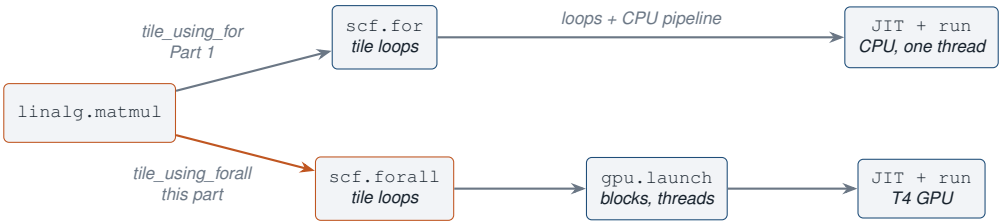
- In Part 1, we tiled Linalg ops into `scf.for` loops: one tile after another, on one CPU thread.

Same Tiling, onto the GPU



- In Part 1, we tiled Linalg ops into `scf.for` loops: one tile after another, on one CPU thread.
- Here we tile the same ops into `scf.forall` loops, whose iterations may run in parallel.

Same Tiling, onto the GPU



- In Part 1, we tiled Linalg ops into `scf.for` loops: one tile after another, on one CPU thread.
- Here we tile the same ops into `scf.forall` loops, whose iterations may run in parallel.
- The schedule also says which loops become the blocks and which the threads, and builds the `gpu.launch`.

Step 1 — The GPU Payloads

The GPU Payloads

session2/linalg_matvec_gpu.mlir:

```
module {
  func.func @matvec(%mat: memref<128x512xf32>, %vec: memref<512xf32>,
                    %out: memref<128xf32>) {
    %mat_u = memref.cast %mat : memref<128x512xf32> to memref<*xf32>
    %vec_u = memref.cast %vec : memref<512xf32> to memref<*xf32>
    %out_u = memref.cast %out : memref<128xf32> to memref<*xf32>
    gpu.host_register %mat_u : memref<*xf32>
    gpu.host_register %vec_u : memref<*xf32>
    gpu.host_register %out_u : memref<*xf32>
    linalg.matvec ins(%mat, %vec : memref<128x512xf32>, memref<512xf32>)
                  outs(%out : memref<128xf32>)
    return
  }
}
```

The GPU Payloads

session2/linalg_matvec_gpu.mlir:

```
module {
  func.func @matvec(%mat: memref<128x512xf32>, %vec: memref<512xf32>,
                    %out: memref<128xf32>) {
    %mat_u = memref.cast %mat : memref<128x512xf32> to memref<*xf32>
    %vec_u = memref.cast %vec : memref<512xf32> to memref<*xf32>
    %out_u = memref.cast %out : memref<128xf32> to memref<*xf32>
    gpu.host_register %mat_u : memref<*xf32>
    gpu.host_register %vec_u : memref<*xf32>
    gpu.host_register %out_u : memref<*xf32>
    linalg.matvec ins(%mat, %vec : memref<128x512xf32>, memref<512xf32>)
                  outs(%out : memref<128xf32>)
    return
  }
}
```

- The host buffers, registered so the GPU can reach them, as in Session 1 Part 2's kernel. Boilerplate: we leave it there.

The GPU Payloads

session2/linalg_matvec_gpu.mlir:

```
module {
  func.func @matvec(%mat: memref<128x512xf32>, %vec: memref<512xf32>,
                    %out: memref<128xf32>) {
    %mat_u = memref.cast %mat : memref<128x512xf32> to memref<*xf32>
    %vec_u = memref.cast %vec : memref<512xf32> to memref<*xf32>
    %out_u = memref.cast %out : memref<128xf32> to memref<*xf32>
    gpu.host_register %mat_u : memref<*xf32>
    gpu.host_register %vec_u : memref<*xf32>
    gpu.host_register %out_u : memref<*xf32>
    linalg.matvec ins(%mat, %vec : memref<128x512xf32>, memref<512xf32>)
                  outs(%out : memref<128xf32>)
    return
  }
}
```

- The host buffers, registered so the GPU can reach them, as in Session 1 Part 2's kernel. Boilerplate: we leave it there.
- The op is Part 1's `linalg.matvec`, unchanged.

Step 2 — Distributing, with a Transform Schedule

Tiling into Parallel Loops

Tiling matvec's `i` by 32, the shape of it:

Part 1: `tile_using_for`

```
scf.for %i = 0 to 128 step 32 {
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
}
```

this part: `tile_using_forall`

```
scf.forall (%b) in (4) {
  %i = affine.apply %b * 32
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
} {mapping = [#gpu.block<x>]}
```

Tiling into Parallel Loops

Tiling matvec's `i` by 32, the shape of it:

Part 1: `tile_using_for`

```
scf.for %i = 0 to 128 step 32 {
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
}
```

this part: `tile_using_forall`

```
scf.forall (%b) in (4) {
  %i = affine.apply %b * 32
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
} {mapping = [#gpu.block<x>]}
```

- `scf.for` runs its iterations one after another, in order.

Tiling into Parallel Loops

Tiling matvec's `i` by 32, the shape of it:

Part 1: `tile_using_for`

```
scf.for %i = 0 to 128 step 32 {
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
}
```

this part: `tile_using_forall`

```
scf.forall (%b) in (4) {
  %i = affine.apply %b * 32
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
} {mapping = [#gpu.block<x>]}
```

- `scf.for` runs its iterations one after another, in order.
- `scf.forall` says its iterations are independent: they may run in any order, or all at once.

Tiling into Parallel Loops

Tiling matvec's `i` by 32, the shape of it:

Part 1: `tile_using_for`

```
scf.for %i = 0 to 128 step 32 {
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
}
```

this part: `tile_using_forall`

```
scf.forall (%b) in (4) {
  %i = affine.apply %b * 32
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
} {mapping = [#gpu.block<x>]}
```

- `scf.for` runs its iterations one after another, in order.
- `scf.forall` says its iterations are independent: they may run in any order, or all at once.
- Its mapping says what runs each iteration on the GPU: here a block, numbered by its block id in x.

Tiling into Parallel Loops

Tiling matvec's `i` by 32, the shape of it:

Part 1: `tile_using_for`

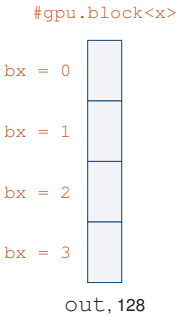
```
scf.for %i = 0 to 128 step 32 {
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
}
```

this part: `tile_using_forall`

```
scf.forall (%b) in (4) {
  %i = affine.apply %b * 32
  %o = memref.subview %out[%i] [32]
  linalg.matvec ins(...) outs(%o)
} {mapping = [#gpu.block<x>]}
```

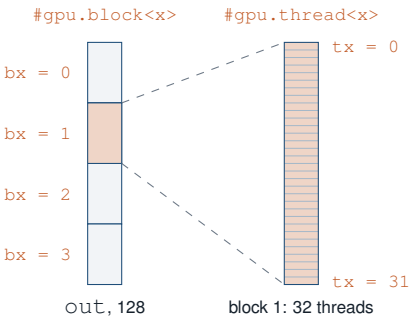
- `scf.for` runs its iterations one after another, in order.
- `scf.forall` says its iterations are independent: they may run in any order, or all at once.
- Its mapping says what runs each iteration on the GPU: here a block, numbered by its block id in x.
- Only a `parallel` loop can be tiled this way: tiles of a reduction would add into the same `out[i]` at once. `tile_using_forall` only warns: `tiling is not thread safe`.

Blocks and Threads, from Two Tilings



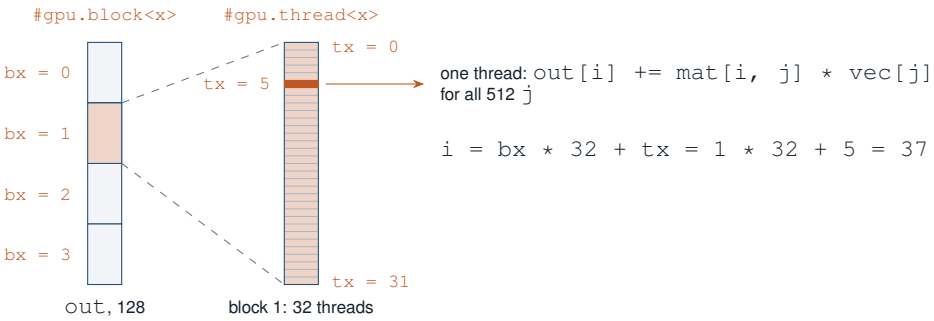
- Tile `i` by 32, mapped to `#gpu.block<x>`: 4 iterations, the grid's 4 blocks, `bx` = 0 . . . 3.

Blocks and Threads, from Two Tilings



- Tile `i` by 32, mapped to `#gpu.block<x>`: 4 iterations, the grid's 4 blocks, `bx` = 0 . . . 3.
- Tile each of those by 1, mapped to `#gpu.thread<x>`: 32 iterations, the 32 threads of each block, `tx` = 0 . . . 31.

Blocks and Threads, from Two Tilings



- Tile `i` by 32, mapped to `#gpu.block<x>`: 4 iterations, the grid's 4 blocks, `bx` = 0 . . . 3.
- Tile each of those by 1, mapped to `#gpu.thread<x>`: 32 iterations, the 32 threads of each block, `tx` = 0 . . . 31.
- Each thread computes one `out[i]`, reducing over `j` itself. Its `i` is Session 1 Part 2's `blockIdx * blockDim + threadIdx`.

Reference: `distribute_rows.mlir`, Step 1 of 4 — The Tilings

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32] (mapping = [#gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1] (mapping = [#gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    // next: build the kernel
    transform.yield
  }
}
```

Reference: `distribute_rows.mlir`, Step 1 of 4 — The Tilings

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32] (mapping = [#gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1] (mapping = [#gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    // next: build the kernel
    transform.yield
  }
}
```

- `match` by interface, as in Part 1: any Linalg op.

Reference: `distribute_rows.mlir`, Step 1 of 4 — The Tilings

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32] (mapping = [#gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1] (mapping = [#gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    // next: build the kernel
    transform.yield
  }
}
```

- `match` by interface, as in Part 1: any Linalg op.
- The blocks: tile `i` by 32 into an `scf.forall`. The mapping has one entry per tiled loop: this one becomes the blocks, by block id in x.

Reference: `distribute_rows.mlir`, Step 1 of 4 — The Tilings

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32] (mapping = [#gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1] (mapping = [#gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    // next: build the kernel
    transform.yield
  }
}
```

- `match` by interface, as in Part 1: any Linalg op.
- The blocks: tile `i` by 32 into an `scf.forall`. The mapping has one entry per tiled loop: this one becomes the blocks, by block id in x.
- The threads: the op inside each block, by 1. Each iteration becomes a thread of that block, by thread id in x.

Reference: `distribute_rows.mlir`, Step 1 of 4 — The Tilings

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32] (mapping = [#gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1] (mapping = [#gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    // next: build the kernel
    transform.yield
  }
}
```

- `match` by interface, as in Part 1: any Linalg op.
- The blocks: tile `i` by 32 into an `scf.forall`. The mapping has one entry per tiled loop: this one becomes the blocks, by block id in x.
- The threads: the op inside each block, by 1. Each iteration becomes a thread of that block, by thread id in x.
- `%block_op` is the op inside the blocks: the first step returns it, and the second tiles it.

Reference: `distribute_rows.mlir`, Step 2 of 4 — Building the Kernel

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(...) {
    // the match, then the block and thread tilings
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 1, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

Reference: `distribute_rows.mlir`, Step 2 of 4 — Building the Kernel

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(...) {
    // the match, then the block and thread tilings
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 1, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Match the function: the next two steps work on the code around the loops.

Reference: `distribute_rows.mlir`, Step 2 of 4 — Building the Kernel

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(...) {
    // the match, then the block and thread tilings
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 1, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Match the function: the next two steps work on the code around the loops.
- `map_forall_to_blocks` puts the block loop into a `gpu.launch`, one block per iteration, and replaces its index with `gpu.block_id x`.

Reference: `distribute_rows.mlir`, Step 2 of 4 — Building the Kernel

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(...) {
    // the match, then the block and thread tilings
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 1, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Match the function: the next two steps work on the code around the loops.
- `map_forall_to_blocks` puts the block loop into a `gpu.launch`, one block per iteration, and replaces its index with `gpu.block_id x`.
- `map_nested_forall_to_threads` does the same for the thread loop, with `gpu.thread_id x`. `block_dims` is the size of each block: 32 threads in x.

Reference: `distribute_rows.mlir`, Step 3 of 4 — Applied to Matvec

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c4, %arg10 = %c1_4, %arg11 = %c1_5)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c1_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_7 = memref.subview %arg1[0] [512] [1] : ...
  %subview_8 = memref.subview %arg2[%0] [32] [1] : ...
  %thread_id_x = gpu.thread_id x
  ...
  %subview_9 = memref.subview %subview[%thread_id_x, 0] [1, 512] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[0] [512] [1] : ...
  %subview_11 = memref.subview %subview_8[%thread_id_x] [1] [1] : ...
  linalg.matvec ins(%subview_9, %subview_10 : ...) outs(%subview_11 : ...)
  gpu.terminator
}
```

Reference: `distribute_rows.mlir`, Step 3 of 4 — Applied to Matvec

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c4, %arg10 = %c1_4, %arg11 = %c1_5)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c1_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_7 = memref.subview %arg1[0] [512] [1] : ...
  %subview_8 = memref.subview %arg2[%0] [32] [1] : ...
  %thread_id_x = gpu.thread_id x
  ...
  %subview_9 = memref.subview %subview[%thread_id_x, 0] [1, 512] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[0] [512] [1] : ...
  %subview_11 = memref.subview %subview_8[%thread_id_x] [1] [1] : ...
  linalg.matvec ins(%subview_9, %subview_10 : ...) outs(%subview_11 : ...)
  gpu.terminator
}
```

- Grid size: `(4, 1, 1)`, block size: `(32, 1, 1)`: the block loop's 4 iterations, and `block_dims`.

Reference: `distribute_rows.mlir`, Step 3 of 4 — Applied to Matvec

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c4, %arg10 = %c1_4, %arg11 = %c1_5)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c1_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_7 = memref.subview %arg1[0] [512] [1] : ...
  %subview_8 = memref.subview %arg2[%0] [32] [1] : ...
  %thread_id_x = gpu.thread_id x
  ...
  %subview_9 = memref.subview %subview[%thread_id_x, 0] [1, 512] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[0] [512] [1] : ...
  %subview_11 = memref.subview %subview_8[%thread_id_x] [1] [1] : ...
  linalg.matvec ins(%subview_9, %subview_10 : ...) outs(%subview_11 : ...)
  gpu.terminator
}
```

- Grid size: `(4, 1, 1)`, block size: `(32, 1, 1)`: the block loop's 4 iterations, and `block_dims`.
- Block `bx` (`%block_id_x`): 32 rows of `mat` and `out`, from row `%0 = bx * 32`.

Reference: `distribute_rows.mlir`, Step 3 of 4 — Applied to Matvec

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c4, %arg10 = %c1_4, %arg11 = %c1_5)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c1_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_7 = memref.subview %arg1[0] [512] [1] : ...
  %subview_8 = memref.subview %arg2[%0] [32] [1] : ...
  %thread_id_x = gpu.thread_id x
  ...
  %subview_9 = memref.subview %subview[%thread_id_x, 0] [1, 512] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[0] [512] [1] : ...
  %subview_11 = memref.subview %subview_8[%thread_id_x] [1] [1] : ...
  linalg.matvec ins(%subview_9, %subview_10 : ...) outs(%subview_11 : ...)
  gpu.terminator
}
```

- Grid size: `(4, 1, 1)`, block size: `(32, 1, 1)`: the block loop's 4 iterations, and `block_dims`.
- Block `bx` (`%block_id_x`): 32 rows of `mat` and `out`, from row `%0 = bx * 32`.
- Thread `tx` (`%thread_id_x`): one row of that slice, row `bx * 32 + tx`.

Reference: `distribute_rows.mlir`, Step 3 of 4 — Applied to Matvec

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c4, %arg10 = %c1_4, %arg11 = %c1_5)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c1_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_7 = memref.subview %arg1[0] [512] [1] : ...
  %subview_8 = memref.subview %arg2[%0] [32] [1] : ...
  %thread_id_x = gpu.thread_id x
  ...
  %subview_9 = memref.subview %subview[%thread_id_x, 0] [1, 512] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[0] [512] [1] : ...
  %subview_11 = memref.subview %subview_8[%thread_id_x] [1] [1] : ...
  linalg.matvec ins(%subview_9, %subview_10 : ...) outs(%subview_11 : ...)
  gpu.terminator
}
```

- Grid size: `(4, 1, 1)`, block size: `(32, 1, 1)`: the block loop's 4 iterations, and `block_dims`.
- Block `bx` (`%block_id_x`): 32 rows of `mat` and `out`, from row `%0 = bx * 32`.
- Thread `tx` (`%thread_id_x`): one row of that slice, row `bx * 32 + tx`.
- Inside, the original `linalg.matvec`, on that one row. Still Linalg: the next step lowers it.

Reference: Step 4 of 4 — Compiling and Execution

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin cubin-features=+ptx78}
```

Reference: Step 4 of 4 — Compiling and Execution

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin cubin-features=+ptx78}
```

- `convert-linalg-to-loops` turns the `linalg.matvec` into the thread's loop over `j`: Session 1 Part 2's matvec kernel, generated.

Reference: Step 4 of 4 — Compiling and Execution

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin cubin-features=+ptx78}
```

- `convert-linalg-to-loops` turns the `linalg.matvec` into the thread's loop over `j`: Session 1 Part 2's matvec kernel, generated.
- Part 1's two passes, for the subviews and their `affine.apply`.

Reference: Step 4 of 4 — Compiling and Execution

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin cubin-features=+ptx78}
```

- `convert-linalg-to-loops` turns the `linalg.matvec` into the thread's loop over `j`: Session 1 Part 2's matvec kernel, generated.
- Part 1's two passes, for the subviews and their `affine.apply`.
- Session 1 Part 2's GPU pipeline: the body compiled for this GPU, the host side rewritten to launch it.

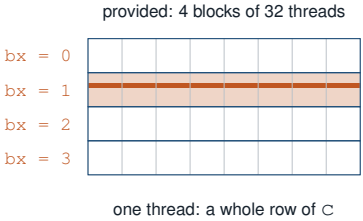
Reference: Step 4 of 4 — Compiling and Execution

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin cubin-features=+ptx78}
```

- `convert-linalg-to-loops` turns the `linalg.matvec` into the thread's loop over `j`: Session 1 Part 2's matvec kernel, generated.
- Part 1's two passes, for the subviews and their `affine.apply`.
- Session 1 Part 2's GPU pipeline: the body compiled for this GPU, the host side rewritten to launch it.
- JIT-compiled as in Session 1 Part 2 — the CUDA runtime library in `shared_libs`, then `engine.initialize()` — and checked against numpy:

```
OK
```

The Same Schedule, on Matmul

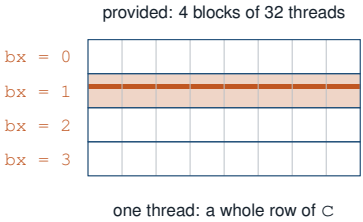


The innermost op, from the same schedule:

```
linalg.matmul ins(%subview_9, %subview_10 : memref<1x512xf32, ...>, memref<512x256xf32, ...>)
              outs(%subview_11 : memref<1x256xf32, ...>)
```

- Unchanged, the schedule tiles only the first loop, `i`, and maps it to x: 4 blocks of 32 threads, as for matvec.

The Same Schedule, on Matmul

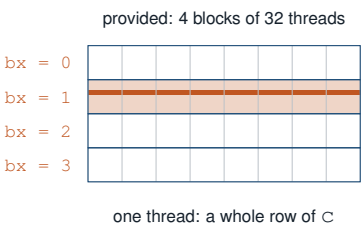


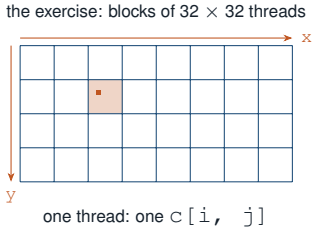
The innermost op, from the same schedule:

```
linalg.matmul ins(%subview_9, %subview_10 : memref<1x512xf32, ...>, memref<512x256xf32, ...>)
              outs(%subview_11 : memref<1x256xf32, ...>)
```

- Unchanged, the schedule tiles only the first loop, `i`, and maps it to x: 4 blocks of 32 threads, as for matvec.
- Each thread gets a 1×256 slice of `C`: a whole row, with `j` and `k` as loops in its body.

The Same Schedule, on Matmul





The innermost op, from the same schedule:

```
linalg.matmul ins(%subview_9, %subview_10 : memref<1x512xf32, ...>, memref<512x256xf32, ...>)
              outs(%subview_11 : memref<1x256xf32, ...>)
```

- Unchanged, the schedule tiles only the first loop, `i`, and maps it to x: 4 blocks of 32 threads, as for matvec.
- Each thread gets a 1×256 slice of `C`: a whole row, with `j` and `k` as loops in its body.
- The exercise: distribute `j` too, so that each thread computes one `C[i, j]`, with the columns on x and the rows on y.

Exercise 6 — Distribute Matmul along Columns Too

Modify `session2/distribute_blocks.mlir`, a copy of `distribute_rows.mlir`, so that each thread computes one `c[i, j]`, in blocks of 32 × 32 threads: the columns (`j`) on x, the rows (`i`) on y.

What is different from matvec:

- Matvec has one parallel loop, `i`. Matmul has two, `i` and `j`; `k` is a reduction, and stays inside each thread.
- One tile size and one mapping entry per loop, in `iterator_types` order: `i` first, then `j`.
- Each block is 32 × 32 threads.

Check after every save — round-trip it:

```
!mlir-opt distribute_blocks.mlir
```

- That checks the schedule is well-formed, not that it does what was intended. For that, run the check under *Matmul, distributed along columns too*: matmul's innermost op must write a 1 × 1 slice of `c`, cut at `[%thread_id_y, %thread_id_x]`.

10 minutes to write it — then we walk through the solution.

Solution — `distribute_blocks.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32, 32] (mapping = [#gpu.block<y>, #gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1, 1] (mapping = [#gpu.thread<y>, #gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 32, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

Solution — `distribute_blocks.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32, 32] (mapping = [#gpu.block<y>, #gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1, 1] (mapping = [#gpu.thread<y>, #gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 32, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Both tilings take `j` as well: `[32, 32]`, then `[1, 1]`.

Solution — `distribute_blocks.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32, 32] (mapping = [#gpu.block<y>, #gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1, 1] (mapping = [#gpu.thread<y>, #gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 32, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Both tilings take `j` as well: `[32, 32]`, then `[1, 1]`.
- The mappings follow the loops: `i` first, on y; `j` second, on x.

Solution — `distribute_blocks.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %block_op, %blocks = transform.structured.tile_using_forall %ops
      tile_sizes [32, 32] (mapping = [#gpu.block<y>, #gpu.block<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %thread_op, %threads = transform.structured.tile_using_forall %block_op
      tile_sizes [1, 1] (mapping = [#gpu.thread<y>, #gpu.thread<x>])
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    %func = transform.structured.match ops{["func.func"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %launch = transform.gpu.map_forall_to_blocks %func generate_gpu_launch
      : (!transform.any_op) -> !transform.any_op
    transform.gpu.map_nested_forall_to_threads %launch block_dims = [32, 32, 1]
      sync_after_distribute = false
      : (!transform.any_op) -> !transform.any_op
    transform.yield
  }
}
```

- Both tilings take `j` as well: `[32, 32]`, then `[1, 1]`.
- The mappings follow the loops: `i` first, on y; `j` second, on x.
- Each block is 32×32 threads.

Solution — `gpu.launch`

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c8, %arg10 = %c4, %arg11 = %c1_4)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c32_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  %block_id_y = gpu.block_id y
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_y)
  %1 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_6 = memref.subview %arg1[0, %1] [512, 32] [1, 1] : ...
  %subview_7 = memref.subview %arg2[%0, %1] [32, 32] [1, 1] : ...
  %thread_id_x = gpu.thread_id x
  %thread_id_y = gpu.thread_id y
  ...
  %subview_8 = memref.subview %subview[%thread_id_y, 0] [1, 512] [1, 1] : ...
  %subview_9 = memref.subview %subview_6[0, %thread_id_x] [512, 1] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[%thread_id_y, %thread_id_x] [1, 1] [1, 1] : ...
  linalg.matmul ins(%subview_8, %subview_9 : ...) outs(%subview_10 : ...)
  gpu.terminator
}
```

Solution — `gpu.launch`

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c8, %arg10 = %c4, %arg11 = %c1_4)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c32_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  %block_id_y = gpu.block_id y
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_y)
  %1 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_6 = memref.subview %arg1[0, %1] [512, 32] [1, 1] : ...
  %subview_7 = memref.subview %arg2[%0, %1] [32, 32] [1, 1] : ...
  %thread_id_x = gpu.thread_id x
  %thread_id_y = gpu.thread_id y
  ...
  %subview_8 = memref.subview %subview[%thread_id_y, 0] [1, 512] [1, 1] : ...
  %subview_9 = memref.subview %subview_6[0, %thread_id_x] [512, 1] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[%thread_id_y, %thread_id_x] [1, 1] [1, 1] : ...
  linalg.matmul ins(%subview_8, %subview_9 : ...) outs(%subview_10 : ...)
  gpu.terminator
}
```

- Grid size: `(8, 4, 1)`, block size: `(32, 32, 1)`: x runs over the columns.

Solution — `gpu.launch`

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c8, %arg10 = %c4, %arg11 = %c1_4)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c32_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  %block_id_y = gpu.block_id y
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_y)
  %1 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_6 = memref.subview %arg1[0, %1] [512, 32] [1, 1] : ...
  %subview_7 = memref.subview %arg2[%0, %1] [32, 32] [1, 1] : ...
  %thread_id_x = gpu.thread_id x
  %thread_id_y = gpu.thread_id y
  ...
  %subview_8 = memref.subview %subview[%thread_id_y, 0] [1, 512] [1, 1] : ...
  %subview_9 = memref.subview %subview_6[0, %thread_id_x] [512, 1] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[%thread_id_y, %thread_id_x] [1, 1] [1, 1] : ...
  linalg.matmul ins(%subview_8, %subview_9 : ...) outs(%subview_10 : ...)
  gpu.terminator
}
```

- Grid size: `(8, 4, 1)`, block size: `(32, 32, 1)`: x runs over the columns.
- Block `(by, bx)` computes the 32 × 32 tile of `c` at `(by * 32, bx * 32)`, reading 32 rows of `a` and 32 columns of `b`.

Solution — `gpu.launch`

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c8, %arg10 = %c4, %arg11 = %c1_4)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c32_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  %block_id_y = gpu.block_id y
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_y)
  %1 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_6 = memref.subview %arg1[0, %1] [512, 32] [1, 1] : ...
  %subview_7 = memref.subview %arg2[%0, %1] [32, 32] [1, 1] : ...
  %thread_id_x = gpu.thread_id x
  %thread_id_y = gpu.thread_id y
  ...
  %subview_8 = memref.subview %subview[%thread_id_y, 0] [1, 512] [1, 1] : ...
  %subview_9 = memref.subview %subview_6[0, %thread_id_x] [512, 1] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[%thread_id_y, %thread_id_x] [1, 1] [1, 1] : ...
  linalg.matmul ins(%subview_8, %subview_9 : ...) outs(%subview_10 : ...)
  gpu.terminator
}
```

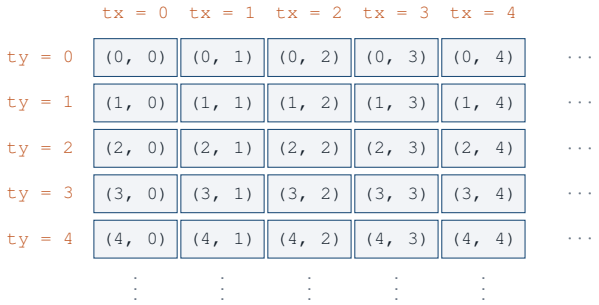
- Grid size: `(8, 4, 1)`, block size: `(32, 32, 1)`: x runs over the columns.
- Block `(by, bx)` computes the 32 × 32 tile of `c` at `(by * 32, bx * 32)`, reading 32 rows of `a` and 32 columns of `b`.
- Thread `(ty, tx)` computes element `[ty, tx]` of that tile, reading one row of `a` and one column of `b`.

Solution — `gpu.launch`

```
gpu.launch blocks(%arg3, %arg4, %arg5) in (%arg9 = %c8, %arg10 = %c4, %arg11 = %c1_4)
           threads(%arg6, %arg7, %arg8) in (%arg12 = %c32, %arg13 = %c32_2, %arg14 = %c1_3) {
  %block_id_x = gpu.block_id x
  %block_id_y = gpu.block_id y
  ...
  %0 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_y)
  %1 = affine.apply affine_map<(d0) -> (d0 * 32)>(%block_id_x)
  %subview = memref.subview %arg0[%0, 0] [32, 512] [1, 1] : ...
  %subview_6 = memref.subview %arg1[0, %1] [512, 32] [1, 1] : ...
  %subview_7 = memref.subview %arg2[%0, %1] [32, 32] [1, 1] : ...
  %thread_id_x = gpu.thread_id x
  %thread_id_y = gpu.thread_id y
  ...
  %subview_8 = memref.subview %subview[%thread_id_y, 0] [1, 512] [1, 1] : ...
  %subview_9 = memref.subview %subview_6[0, %thread_id_x] [512, 1] [1, 1] : ...
  %subview_10 = memref.subview %subview_7[%thread_id_y, %thread_id_x] [1, 1] [1, 1] : ...
  linalg.matmul ins(%subview_8, %subview_9 : ...) outs(%subview_10 : ...)
  gpu.terminator
}
```

- Grid size: `(8, 4, 1)`, block size: `(32, 32, 1)`: x runs over the columns.
- Block `(by, bx)` computes the 32 × 32 tile of `c` at `(by * 32, bx * 32)`, reading 32 rows of `a` and 32 columns of `b`.
- Thread `(ty, tx)` computes element `[ty, tx]` of that tile, reading one row of `a` and one column of `b`.
- Lowered, that is 512 iterations of `k`. Run, and checked against numpy: `OK`

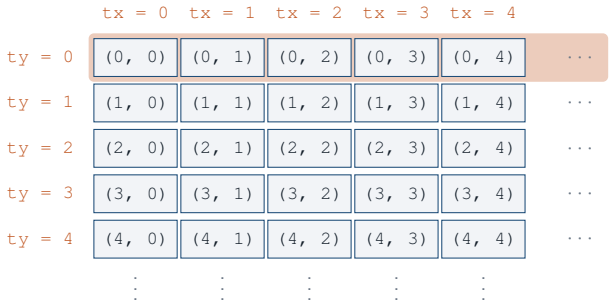
Which Thread Computes Which Element



a block's 32 × 32 tile of `C`:
each cell is the element `(i, j)`

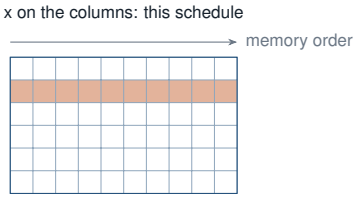
- The y ids pick the row of `c`, `by * 32 + ty`; the x ids pick the column, `bx * 32 + tx`. The schedule said so.

Which Thread Computes Which Element



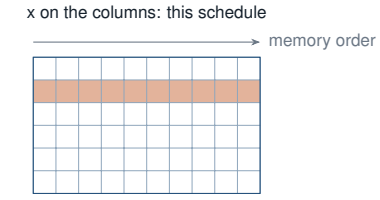
- The y ids pick the row of `C`, `by * 32 + ty`; the x ids pick the column, `bx * 32 + tx`. The schedule said so.
- A warp is 32 threads with consecutive `tx` and the same `ty`: here, 32 elements along a row of `C`.

Why x on the Columns: Coalescing



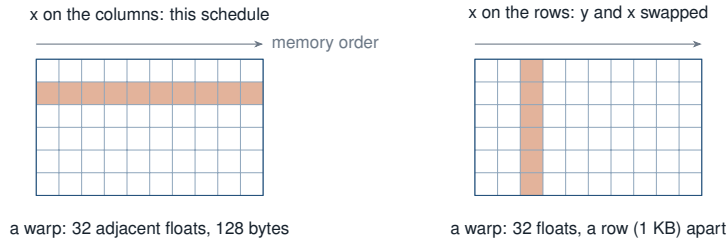
- `c` is row-major: `c[i, j]` and `c[i, j + 1]` are adjacent in memory.

Why x on the Columns: Coalescing



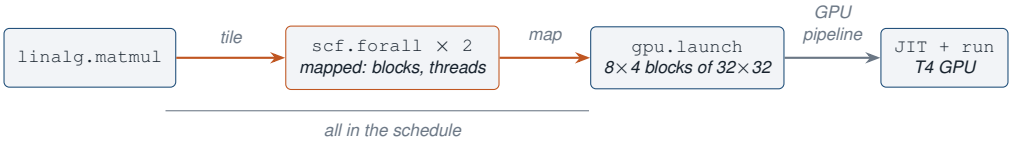
- `c` is row-major: `c[i, j]` and `c[i, j + 1]` are adjacent in memory.
- A warp's loads and stores are coalesced into one transaction when its 32 addresses are adjacent. With x on `j`, a warp writes 32 adjacent floats of `c`, and reads 32 of `b`.

Why x on the Columns: Coalescing

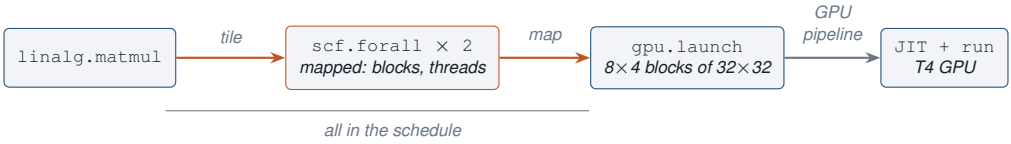


- `c` is row-major: `c[i, j]` and `c[i, j + 1]` are adjacent in memory.
- A warp's loads and stores are coalesced into one transaction when its 32 addresses are adjacent. With x on `j`, a warp writes 32 adjacent floats of `c`, and reads 32 of `b`.

Recap

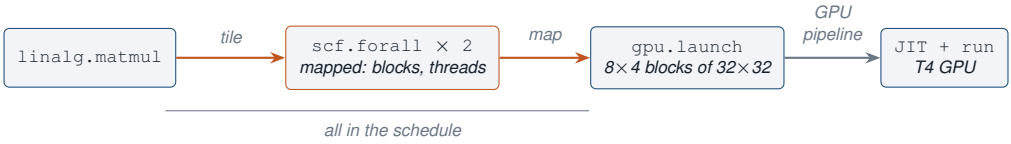

linalg.matmul
tile
scf.forall × 2
mapped: blocks, threads
map
gpu.launch
8×4 blocks of 32×32
GPU
pipeline
JIT + run
T4 GPU
all in the schedule

Recap



- The same tiling as Part 1, into `scf.forall` loops, each mapped onto the GPU: the outer one to blocks, the nested one to threads.

Recap



- The same tiling as Part 1, into `scf.forall` loops, each mapped onto the GPU: the outer one to blocks, the nested one to threads.
- The schedule states the mapping, so it also decides how memory is accessed: x on the columns, and each warp touches adjacent floats.